

Autoencoders

Un *autoencoder* es una red neuronal completamente conectada diseñada para codificar los datos de entrada en una representación comprimida y útil para después decodificarla de nuevo, reconstruyendo los datos de entrada originales con la menor alteración posible. En este capítulo se introduce el concepto de autoencoder y se describen algunas de sus aplicaciones.

La arquitectura consta de dos módulos conectados secuencialmente: el *encoder*, que transforma los datos de entrada en un espacio de menor dimensión, y el *decoder*, que reconstruye los datos originales a partir de esa representación comprimida. Los autoencoders se entrenan para descubrir características relevantes que describen la distribución subyacente de los datos, dichas características conforman la representación latente y concentran la información esencial del conjunto de entrada. Los autoencoders pertenecen a un conjunto de redes neuronales artificiales cuyo entrenamiento no necesita de etiquetas, a diferencia de las redes completamente conectadas utilizadas para clasificación o regresión, el entrenamiento utiliza exclusivamente los datos de entrada, sin etiquetas, y emplea esos mismos datos como objetivo para minimizar el error de reconstrucción.

Para definir matemáticamente un autoencoder, consideramos al conjunto $X = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}\}$ tal que $X \subset \mathbb{R}^n$, donde m es el número total de muestras de entrenamiento. Sean $\mathcal{E}$ y $\mathcal{D}$ dos funciones arbitrarias tales que $\mathcal{E} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ y $\mathcal{D} : \mathbb{R}^p \rightarrow \mathbb{R}^n$ donde $p < n$. Tenemos entonces que, matemáticamente, el autoencoder transforma un vector de entrada $\mathbf{x}^{(i)} \in \mathbb{R}^n$ en un vector de salida $\mathcal{D}[\mathcal{E}(\mathbf{x}^{(i)})] \in \mathbb{R}^n$. Utilizando la función de costo error cuadrático medio, podemos calcular el promedio del error Δ entre el vector de entrada $\mathbf{x}^{(i)}$ y su reconstrucción $\mathcal{D}[\mathcal{E}(\mathbf{x}^{(i)})]$ para todas las muestras del conjunto X .

$$\Delta = \frac{1}{mn} \sum_{i=1}^m \sum_{d=1}^n \left(x_d^{(i)} - \mathcal{D}[\mathcal{E}(\mathbf{x}^{(i)})]_d \right)^2 \quad (1)$$

Donde $x_d^{(i)}$ corresponde a la entrada del vector $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_d^{(i)}, \dots, x_n^{(i)})$ en la posición d , mientras que $\mathcal{D}[\mathcal{E}(\mathbf{x}^{(i)})]_d$ corresponde a la entrada del vector $\mathcal{D}[\mathcal{E}(\mathbf{x}^{(i)})]$ en la posición d .

El problema del autoencoder es encontrar las funciones $\mathcal{E}$ y $\mathcal{D}$ que minimizan el promedio del error entre los datos de entrada y su reconstrucción. En el contexto de redes neuronales artificiales, las funciones $\mathcal{E}$ y $\mathcal{D}$ son redes neuronales completamente conectadas. Llamaremos codificador o *encoder* a la red representada por la función $\mathcal{E}$, la cual tendrá n neuronas de entrada y p neuronas de salida, mientras que llamaremos decodificador o *decoder* a la red representada por la función $\mathcal{D}$, compuesta por p neuronas de entrada y n neuronas de salida.

Entrenamiento de un autoencoder

A pesar de que podemos considerar los autoencoders como una arquitectura compuesta por dos redes neuronales separadas, el entrenamiento se realiza como si se tratara de una sola red completamente conectada, donde el vector de salida del encoder se utiliza como vector de entrada del decoder, haciendo que ambas redes compartan una misma capa, es decir, la capa de salida del codificador corresponde a la capa de entrada del decodificador, de esta manera ambas redes conforman una sola. El entrenamiento del autoencoder se divide en dos fases principales: propagación hacia adelante e inferencia y propagación hacia atrás o retropropagación.

La propagación hacia adelante en un autoencoder sigue el mismo principio que en las redes neuronales tradicionales, pero con la particularidad de que el objetivo es reconstruir la entrada original. Este proceso se desarrolla en dos etapas secuenciales: la codificación y la decodificación.

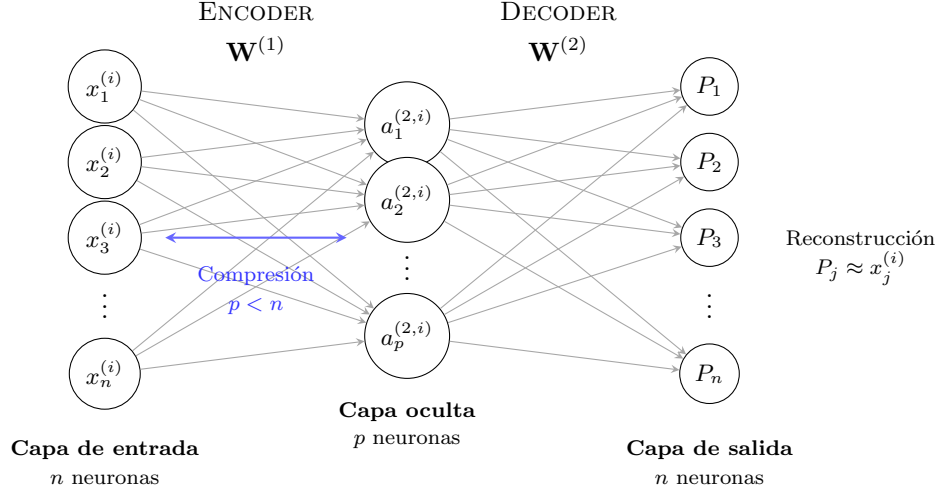


Figure 1: Arquitectura de un autoencoder de tres capas mostrando el encoder (capas 1-2) y decoder (capas 2-3). La capa oculta contiene la representación comprimida de dimensión $p < n$.

Como ejemplo, consideremos un autoencoder de tres capas tal como la que se observa en la [Figura 1](#), el autoencoder consta de una capa de entrada con n neuronas, una capa oculta con p neuronas que representa la codificación comprimida, y una capa de salida con n neuronas que produce la reconstrucción. Siguiendo la lógica explicada en el párrafo anterior, el codificador será una red neuronal sin capas ocultas conformada por las capas 1 y 2. Para calcular la salida del codificador, realizamos la propagación hacia adelante, partiendo de los valores de entrada para calcular la salida de la capa 2 (o capa de salida del codificador):

$$z_{k_1}^{(2,i)} = \sum_{j_1=1}^n \omega_{j_1,k_1}^{(1)} x_{j_1}^{(i)} + b_{k_1}^{(1)} \quad (2)$$

donde $j_1 \in [1, n]$ indexa las neuronas de la capa de entrada, $k_1 \in [1, p]$ indexa las neuronas de la capa oculta, y $b_{k_1}^{(1)}$ es el término de sesgo para la neurona k_1 de la capa oculta. Los pesos $\omega_{j_1,k_1}^{(1)}$ conectan la neurona j_1 de la capa de entrada con la neurona k_1 de la capa oculta. Las activaciones de la capa de salida del codificador se obtienen aplicando una función de activación $g(\cdot)$ a cada uno de los resultados de las sumatorias $z_{k_1}^{(2,i)}$:

$$a_{k_1}^{(2,i)} = g(z_{k_1}^{(2,i)}) \quad (3)$$

El vector resultante $\mathbf{a}^{(2,i)} = [a_1^{(2,i)}, a_2^{(2,i)}, \dots, a_p^{(2,i)}]^T$ constituye la representación comprimida o codificación del vector de entrada original en un espacio de menor dimensión. El decodificador será entonces una red neuronal sin capas ocultas conformada por las capas 2 y 3. Para calcular la salida del decodificador, realizamos nuevamente la propagación hacia adelante, utilizando los valores de salida de la capa 2 (salida del codificador) como entrada para calcular la salida de la capa 3 (o capa de salida del decodificador):

$$z_{k_2}^{(3,i)} = \sum_{j_2=1}^p \omega_{j_2,k_2}^{(2)} a_{j_2}^{(2,i)} + b_{k_2}^{(2)} \quad (4)$$

donde $j_2 \in [1, p]$ indexa las neuronas de la capa oculta, $k_2 \in [1, n]$ indexa las neuronas de la capa de salida, y $b_{k_2}^{(2)}$ es el término de sesgo para la neurona k_2 de la capa de salida. Los pesos $\omega_{j_2,k_2}^{(2)}$ conectan la

neurona j_2 de la capa oculta con la neurona k_2 de la capa de salida. Las activaciones de la capa de salida, que representan la reconstrucción de los datos de entrada originales, se obtienen como:

$$a_{k_2}^{(3,i)} = g(z_{k_2}^{(3,i)}) \quad (5)$$

Ahora podemos expresar formalmente las funciones $\mathcal{E}$ y $\mathcal{D}$ en términos de las activaciones calculadas. La función codificadora $\mathcal{E} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ transforma el vector de entrada $\mathbf{x}^{(i)} \in \mathbb{R}^n$ en la representación comprimida de la capa oculta:

$$\mathcal{E}(\mathbf{x}^{(i)}) = \begin{bmatrix} a_1^{(2,i)} \\ a_2^{(2,i)} \\ \vdots \\ a_p^{(2,i)} \end{bmatrix} = \begin{bmatrix} g(z_1^{(2,i)}) \\ g(z_2^{(2,i)}) \\ \vdots \\ g(z_p^{(2,i)}) \end{bmatrix} \quad (6)$$

De manera similar, la función decodificadora $\mathcal{D} : \mathbb{R}^p \rightarrow \mathbb{R}^n$ transforma la representación comprimida de vuelta al espacio original. En muchos autoencoders con objetivo de reconstrucción mediante error cuadrático medio, se adopta una activación lineal (identidad) en la capa de salida. En ese caso, g en la capa de salida puede interpretarse como la función identidad.

$$\mathcal{D}(\mathcal{E}(\mathbf{x}^{(i)})) = \begin{bmatrix} a_1^{(3,i)} \\ a_2^{(3,i)} \\ \vdots \\ a_n^{(3,i)} \end{bmatrix} = \begin{bmatrix} g(z_1^{(3,i)}) \\ g(z_2^{(3,i)}) \\ \vdots \\ g(z_n^{(3,i)}) \end{bmatrix} = \begin{bmatrix} P_1^{(i)} \\ P_2^{(i)} \\ \vdots \\ P_n^{(i)} \end{bmatrix} \quad (7)$$

donde cada elemento $a_{k_2}^{(3,i)} = P_{k_2}^{(i)}$ para $k_2 \in [1, n]$, representa la reconstrucción de la componente k_2 -ésima del vector de entrada original. Por lo tanto, la reconstrucción completa del autoencoder se expresa como:

$$\mathcal{D}[\mathcal{E}(\mathbf{x}^{(i)})] = \mathbf{P}^{(i)} = [P_1^{(i)}, P_2^{(i)}, \dots, P_n^{(i)}]^T \quad (8)$$

La fase de retropropagación se realiza de la manera usual siguiendo la idea de que un autoencoder se entrena como una sola red neuronal artificial.

El cuello de botella

La restricción dimensional $p < n$ constituye el elemento fundamental que define la arquitectura y funcionalidad de un autoencoder. Esta configuración crea lo que se conoce como cuello de botella, donde la información de entrada debe pasar forzosamente a través de una representación de menor dimensionalidad antes de ser reconstruida.

La necesidad del cuello de botella surge de la naturaleza misma del problema que buscan resolver los autoencoders. Si permitimos que $p = n$, la red tendría la capacidad teórica de aprender la función identidad $\mathcal{D}[\mathcal{E}(\mathbf{x})] = \mathbf{x}$ de manera trivial, simplemente copiando cada entrada directamente a la salida correspondiente a través de la capa oculta. En este escenario, el autoencoder no estaría forzado a aprender una representación comprimida y significativa de los datos, sino que podría memorizar cada muestra individualmente sin extraer patrones o estructuras subyacentes. Consideremos tres casos posibles para la dimensionalidad de la capa oculta y sus consecuencias:

Caso 1: $p > n$ La función $\mathcal{E} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ mapea hacia un espacio de mayor dimensión, proporcionando capacidad excesiva que permite memorizar completamente los datos sin extraer patrones significativos. Esta configuración contradice el propósito fundamental de compresión y puede generar representaciones redundantes.

Caso 2: $p = n$: En este escenario, el autoencoder puede aprender a reproducir exactamente la entrada en la salida, es decir, implementar una transformación identidad. Aunque en teoría el error de reconstrucción podría llegar a ser nulo, la red no realiza compresión ni extrae representaciones latentes más útiles, limitándose únicamente a copiar los datos de entrada.

Caso 3: $p < n$: La restricción dimensional $p < n$ obliga al autoencoder a identificar y preservar únicamente las características más relevantes de los datos, descartando información redundante. Esta compresión forzada actúa como regularización implícita, resultando en representaciones densas donde cada neurona de la capa oculta contribuye significativamente a la reconstrucción y captura patrones esenciales de la estructura subyacente de los datos.

La elección de p determina directamente el grado de compresión y, por tanto, el balance entre capacidad representacional y generalización. Un valor de p demasiado pequeño puede resultar en pérdida excesiva de información y reconstrucciones de baja calidad, mientras que un valor demasiado grande puede reducir la efectividad de la compresión. La selección óptima de p constituye un hiperparámetro crucial que debe ajustarse según las características específicas de los datos y el objetivo de la aplicación.

Ejemplo Resuelto

Para ilustrar los conceptos fundamentales de compresión en autoencoders, presentamos un ejemplo práctico que extiende la comprensión del cuello de botella a un caso tridimensional. Implementamos un autoencoder con arquitectura $[3, 2, 3]$ que procesa 500 puntos uniformemente distribuidos en una esfera unitaria de $\mathbb{R}^3$. La función $\mathcal{E} : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ comprime la superficie esférica tridimensional a una representación bidimensional, mientras que $\mathcal{D} : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ intenta reconstruir los puntos originales.

El autoencoder se entrena con tasa de aprendizaje $\alpha = 0.2$ durante 200 épocas utilizando mini-batches de tamaño 50 utilizando gradiente descendente como optimizador. El encoder utiliza función de activación lineal, que mapea las coordenadas (x, y, z) de la esfera al espacio latente bidimensional. El decoder emplea activación lineal para la reconstrucción. Esta configuración resulta en las transformaciones:

$$\mathcal{E}(\mathbf{x}) = \mathbf{x} \cdot \mathbf{W}^{(1)} + \mathbf{b}^{(1)} \quad (9)$$

$$\mathcal{D}(\mathcal{E}(\mathbf{x})) = \mathcal{E}(\mathbf{x}) \cdot \mathbf{W}^{(2)} + \mathbf{b}^{(2)} \quad (10)$$

donde $\mathbf{W}^{(1)} \in \mathbb{R}^{3 \times 2}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{2 \times 3}$, $\mathbf{b}^{(1)} \in \mathbb{R}^2$, y $\mathbf{b}^{(2)} \in \mathbb{R}^3$ son los vectores de sesgo correspondientes.

La [Figura 2](#) muestra cómo el encoder transforma la superficie esférica tridimensional en una representación bidimensional. La colorización por coordenada z revela que el autoencoder aprende a preservar cierta estructura geométrica de la esfera original, mapeando regiones de altitud similar a zonas próximas en el espacio latente. Sin embargo, la proyección de la superficie de la esfera inmersa en un espacio tridimensional $\mathbb{R}^3$ a un espacio plano $\mathbb{R}^2$ introduce inevitablemente distorsiones geométricas.

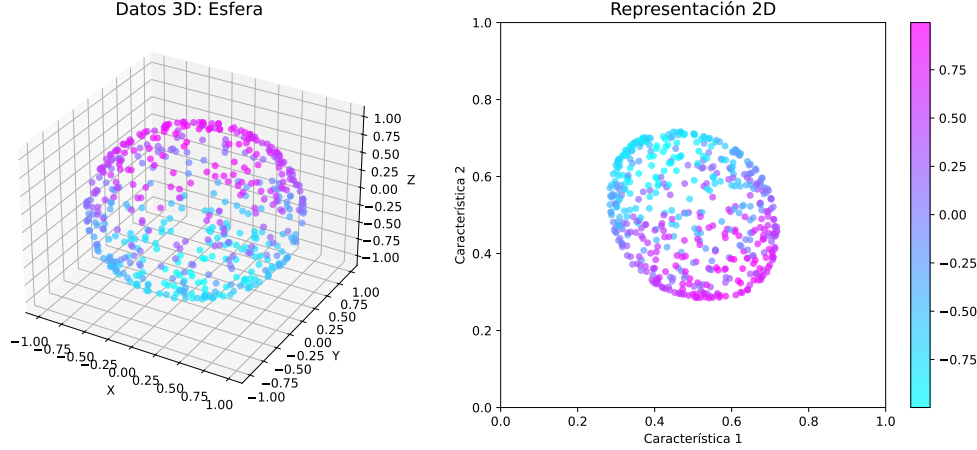


Figure 2: Mapeo de esfera 3D a representación latente 2D. (Izquierda) 500 puntos distribuidos uniformemente en esfera unitaria, coloreados por coordenada z . (Derecha) Representación comprimida en espacio latente bidimensional.

La [Figura 3](#) evidencia las limitaciones fundamentales del proceso de compresión. La superficie esférica original, definida matemáticamente por la ecuación $x^2 + y^2 + z^2 = 1$, no puede ser preservada exactamente tras la compresión a dos dimensiones. Desde una perspectiva funcional, la reconstrucción perfecta $\mathcal{D}[\mathcal{E}(\mathbf{x})] = \mathbf{x}$ para todo $\mathbf{x}$ es matemáticamente imposible cuando $p < n$. La función codificadora $\mathcal{E} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ debe asignar cada punto del dominio $\mathbb{R}^n$ a un único punto en el codominio $\mathbb{R}^p$. Sin embargo, cuando $p < n$, el codominio tiene menor dimensionalidad que el dominio. Por lo tanto, si intentamos distribuir puntos de $\mathbb{R}^n$ en el espacio más pequeño $\mathbb{R}^p$, necesariamente existen al menos dos puntos distintos $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^n$ tales que $\mathcal{E}(\mathbf{x}_1) = \mathcal{E}(\mathbf{x}_2)$. Esto significa que $\mathcal{E}$ no es inyectiva.

La consecuencia directa de la no-inyectividad es que la función decodificadora $\mathcal{D}$ recibe la misma representación $\mathcal{E}(\mathbf{x}_1) = \mathcal{E}(\mathbf{x}_2)$ para dos entradas diferentes. Como $\mathcal{D}$ es una función, debe producir una única salida para cada entrada, pero no puede decidir si reconstruir $\mathbf{x}_1$ o $\mathbf{x}_2$. Por tanto, al menos uno de estos puntos no puede ser reconstruido perfectamente, garantizando que $\mathcal{D}[\mathcal{E}(\mathbf{x}_i)] \neq \mathbf{x}_i$ para algún $i \in \{1, 2\}$. Esta pérdida de información es inevitable y cuantifica el error mínimo de reconstrucción inherente a cualquier autoencoder con $p < n$.

Esta limitación matemática es precisamente lo que hace que la configuración $n > p$ sea ideal para los autoencoders. La imposibilidad de reconstrucción perfecta fuerza al autoencoder a aprender una representación no trivial de los datos de entrada. Durante el entrenamiento, la red debe identificar y preservar únicamente las características más importantes y representativas, descartando información redundante o menos relevante. Este proceso de selección forzada resulta en representaciones comprimidas que capturan la estructura esencial subyacente de los datos, en lugar de simplemente memorizarlos.

Ejercicios

1. Implemente un autoencoder con $n = 3$ y $p = 2$ para visualizar el proceso de compresión.
 - (a) Genere 500 puntos que formen un cubo en el espacio (x_1, x_2, x_3)
 - (b) Entrene un autoencoder con arquitectura $[3, 2, 3]$
 - (c) Visualice: datos originales, representación 2D, y reconstrucción
 - (d) Calcule el error promedio usando la Ecuación 1

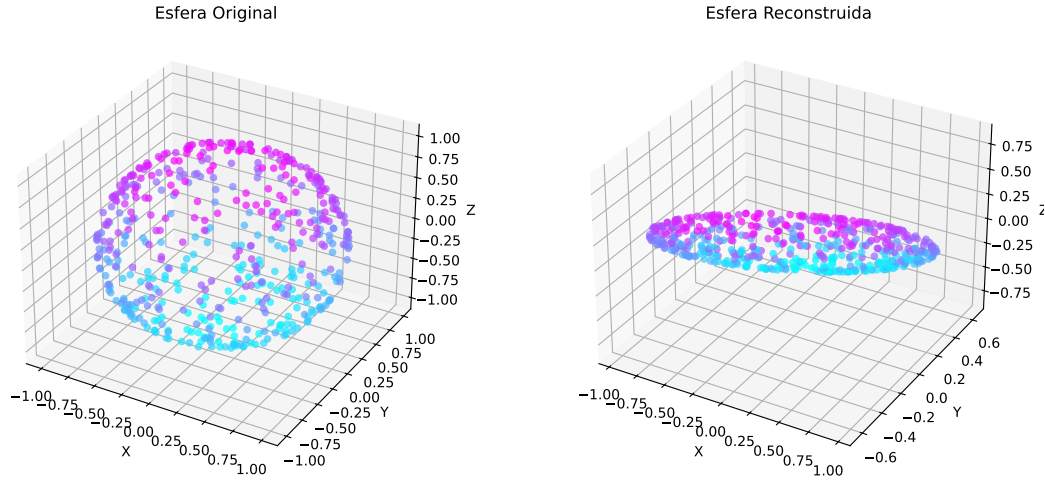


Figure 3: Comparación entre esfera original y reconstruida. (Izquierda) Esfera unitaria original con 500 puntos. (Derecha) Reconstrucción obtenida mediante el proceso completo $\mathcal{D}[\mathcal{E}(\mathbf{x})]$.

2. Compare cómo diferentes tipos de datos afectan el error de reconstrucción Δ , para ello, entrene autoencoders idénticos $[10, 3, 10]$ en tres conjuntos de datos.

- (a) Dataset A: 10 valores que varían suavemente de 0 a 1
- (b) Dataset B: Dataset A + ruido gaussiano ($\sigma = 0.1$)
- (c) Dataset C: Señal cuadrada con valores 0 y 1

Grafique la evolución del error durante 1000 épocas para cada dataset y responda ¿Por qué algunos datos son más difíciles de reconstruir?